

Base de Datos del Proyecto ALERTSYNC en MongoDB Atlas

Materia: Proyecto Integrador
Proyecto: ALERTSYNC — Sistema de seguridad personal
Fecha: 24 de julio de 2026

1. Objetivo

Documentar el diseño e implementación de la base de datos NoSQL del proyecto ALERTSYNC, alojada en **MongoDB Atlas** (nube), cubriendo: el modelo de datos, los archivos de carga inicial, la evidencia de su despliegue en Atlas, el procedimiento de respaldo de la información, los índices definidos para soportar las consultas del sistema, y las medidas adoptadas para garantizar la calidad e integridad de los datos almacenados.

2. Materiales

Herramienta	Uso en el proyecto
MongoDB Atlas	Servicio de base de datos NoSQL administrado en la nube; aloja el clúster Memo donde vive la base de datos alertsync .
MongoDB Database Tools (mongodump, mongoexport, mongoimport, mongosh)	Utilerías de línea de comandos para inspeccionar, respaldar y cargar datos en la base de datos.
Laravel 13 + mongodb/laravel-mongodb	Framework backend (PHP) que conecta la aplicación a MongoDB Atlas mediante Eloquent, usando el driver oficial de MongoDB para PHP.
Vue 3	Frontend de la aplicación web, consume la API REST del backend.
Postman / curl	Pruebas directas de los endpoints de la API que leen y escriben en la base de datos.

3. Marco Teórico

3.1 Bases de datos NoSQL orientadas a documentos

A diferencia de una base de datos relacional (SQL), donde la información se organiza en tablas con un esquema fijo y relaciones mediante llaves foráneas, una base de datos **NoSQL orientada a documentos** como MongoDB almacena la información en **documentos** con formato similar a JSON (internamente BSON — Binary JSON), agrupados en **colecciones**. Cada documento puede tener una estructura ligeramente distinta a otros documentos de la misma colección, lo que da flexibilidad para modelar datos que evolucionan con el tiempo, como ocurrió con los campos que se fueron agregando a la colección **users** conforme creció el proyecto (por ejemplo **username**, **devices**, **device_names**).

3.2 MongoDB Atlas

MongoDB Atlas es el servicio de base de datos **como servicio (DBaaS)** ofrecido por MongoDB Inc. Permite desplegar un clúster de MongoDB completamente administrado en la nube (alta disponibilidad, respaldos automáticos, monitoreo, escalamiento), sin necesidad de instalar ni mantener un servidor de base de datos propio. La aplicación se conecta a Atlas mediante una cadena de conexión con el esquema **mongodb+srv://**, que utiliza

registros DNS de tipo SRV para descubrir automáticamente los nodos del clúster (a diferencia de una instalación local, que usaría `mongodb://localhost:27017`).

3.3 Índices en MongoDB

Un índice es una estructura de datos auxiliar (normalmente un árbol B) que MongoDB mantiene sobre uno o más campos de una colección, para evitar tener que recorrer (`COLLSCAN`) todos los documentos cada vez que se ejecuta una consulta. Sin el índice adecuado, una consulta que hoy es rápida con pocos documentos se vuelve progresivamente más lenta conforme crece la colección. MongoDB también soporta **índices únicos** (garantizan que ningún otro documento repita el valor de ese campo) e **índices dispersos (sparse)**, que solo indexan los documentos donde el campo realmente existe.

3.4 Respaldos (backups)

Un respaldo es una copia de los datos que permite restaurar el sistema ante una pérdida o corrupción de información. En MongoDB, el mecanismo estándar de respaldo lógico es **mongodump**, que exporta cada colección a un archivo binario (.bson) junto con sus metadatos; su contraparte **mongorestore** reconstruye la base de datos exactamente como estaba. De forma complementaria, **mongoexport** permite obtener cada colección como texto plano en formato JSON, útil para inspección humana o para intercambiar datos con otros sistemas.

4. Desarrollo

4.1 Modelo de la base de datos NoSQL

La base de datos `alertsync` está compuesta por **6 colecciones**. A continuación se documenta el esquema de cada una, tal como está definido en los modelos Eloquent del backend (`app/Models/*.php`):

Colección `users`

Campo	Tipo	Descripción
<code>_id</code>	ObjectId	Identificador único (autogenerado)
<code>name</code>	string	Nombre completo
<code>email</code>	string	Correo de contacto (puede repetirse entre administradores)
<code>username</code>	string null	Identificador único de inicio de sesión, solo para administradores
<code>phone</code>	string null	Teléfono
<code>password</code>	string (hash bcrypt)	Contraseña cifrada, nunca en texto plano
<code>plan</code>	string	Demo, Básico o Premium
<code>plan_expires_at</code>	date null	Vigencia del plan pagado
<code>next_plan</code>	string null	Plan programado para el siguiente ciclo
<code>devices</code>	objeto embebido	<code>{ watch: bool, alexa: bool }</code> — estado de vinculación de dispositivos
<code>device_names</code>	objeto embebido	Nombre del dispositivo vinculado (real vía Bluetooth o simulado)
<code>role</code>	string	user o admin
<code>status</code>	string	pending (sin activar) o active
<code>activation_code</code> , <code>activation_token</code>	string (hash)	Código OTP y token de activación de cuenta, con expiración
<code>password_reset_code</code> , <code>password_reset_token</code>	string (hash)	Igual, para el flujo de "olvidé mi contraseña"
<code>payment_card_last_four</code> , <code>card_name</code> , <code>card_expiry</code>	string	Solo últimos 4 dígitos de la tarjeta — nunca el número completo ni el CVV
<code>auto_renew</code>	boolean	Renovación automática del plan

Colección `contacts`

Campo	Tipo	Descripción
<code>_id</code>	ObjectId	Identificador único
<code>user_id</code>	string	Referencia al <code>_id</code> del usuario dueño del contacto (relación por referencia)
<code>name</code> , <code>phone</code> , <code>email</code> , <code>relationship</code>	string	Datos del contacto de emergencia
<code>notify_sms</code> , <code>notify_call</code> , <code>notify_email</code>	boolean	Canales de notificación habilitados
<code>priority</code>	integer	Orden de notificación
<code>verified</code>	boolean	Si el contacto ya confirmó su correo
<code>verification_token</code>	string (hash, oculto)	Token de verificación de un solo uso

Colección **alerts**

Campo	Tipo	Descripción
<code>_id</code>	ObjectId	Identificador único
<code>user_id</code>	string	Referencia al usuario que activó la alerta
<code>source</code>	string	Origen de la alerta (dashboard-premium, smartwatch, etc.)
<code>status</code>	string	Estado del envío
<code>latitude</code> , <code>longitude</code>	float	Ubicación GPS al momento de la alerta
<code>message</code>	string	Mensaje de la alerta
<code>contacts_notified</code>	array	Lista embebida de contactos notificados
<code>metadata</code>	objeto embebido	IP, user agent y otros datos de contexto

Colección **payments**

Campo	Tipo	Descripción
<code>_id</code>	ObjectId	Identificador único
<code>user_id</code>	string	Referencia al usuario que realizó el pago
<code>plan</code>	string	Plan contratado
<code>amount</code>	float	Monto cobrado
<code>currency</code>	string	MXN
<code>card_last_four</code> , <code>card_name</code>	string	Datos mínimos de la tarjeta usada
<code>status</code>	string	completed, refunded, etc.

Colección **plans**

Campo	Tipo	Descripción
<code>_id</code>	ObjectId	Identificador único
<code>plan_id</code>	string	basico o premium (identificador estable usado por la aplicación)
<code>name</code>	string	Nombre visible del plan
<code>price_amount</code>	integer	Precio mensual en MXN
<code>max_contacts</code>	integer	Límite de contactos de emergencia permitidos
<code>features</code>	array de string	Lista de características mostradas en la web pública

Colección **personal_access_tokens**

Colección administrada automáticamente por **Laravel Sanctum** (paquete de autenticación), donde se guardan los tokens de sesión activos de cada usuario. No se documenta como parte del modelo de negocio porque es infraestructura de autenticación, no datos del dominio de la aplicación.

Relaciones entre colecciones: al ser una base de datos de documentos (no relacional), ALERTSYNC modela las relaciones **por referencia** en vez de por llave foránea: `contacts.user_id` , `alerts.user_id` y `payments.user_id` almacenan el `_id` del documento correspondiente en `users` . La integridad de estas referencias se garantiza a nivel de aplicación: `Admin\\UserController::destroy()` borra en cascada los

contactos, alertas y pagos asociados antes de eliminar al usuario, ya que MongoDB no ofrece llaves foráneas ni `ON DELETE CASCADE` de forma nativa.

4.2 Archivos JSON de carga de datos iniciales

De las 6 colecciones del proyecto, solo **plans** (y opcionalmente `site_content`, el contenido editable de la página pública) necesitan datos precargados de fábrica; el resto (`users`, `contacts`, `alerts`, `payments`) se llenan orgánicamente conforme las personas usan la aplicación.

Se generaron los siguientes archivos, listos para cargarse con `mongoimport` (incluidos en la carpeta `seed/` de esta entrega):

- **seed/plans.seed.json** — catálogo inicial de los planes Básico (\$100 MXN/mes, 5 contactos) y Premium (\$200 MXN/mes, 15 contactos).
- **seed/site_content.seed.json** — textos de fábrica de la página pública (hero, características, preguntas frecuentes).

```
mongoimport --uri="<cadena-de-conexion-atlas>" --collection=plans --jsonArray --file=plans.seed.json
```

Vale la pena señalar una decisión de diseño relevante para la calidad de los datos: la aplicación **no depende** de que estos documentos existan de antemano. Si `plans` está vacía, el backend (`Plan::getOrDefault()`) sirve estos mismos valores por defecto directamente desde el código, y solo los persiste en la base de datos hasta que un administrador edita un plan por primera vez desde el panel — evitando una base de datos "a medio sembrar".

4.3 Evidencia de que la base de datos está creada en MongoDB Atlas

La base de datos `alertsync` está alojada en el clúster **Memo** de MongoDB Atlas. La cadena de conexión usada por el backend (`backend/.env`, credenciales ocultas por seguridad) es:

```
MONGODB_URI=mongodb+srv://guillermoamescua23s:.....@memo.5kbmqk.mongodb.net/alertsync?retryWrites=true&appName=alertsync
MONGODB_DATABASE=alertsync
```

El prefijo `mongodb+srv://` junto con el dominio `*.mongodb.net` es la firma característica de un clúster alojado en Atlas (no es posible obtenerla con una instalación local de MongoDB).

Como evidencia adicional, se consultó directamente el clúster con `mongoosh` para listar las colecciones reales:

```
> db.getCollectionNames()
[
  'users', 'alerts', 'contacts',
  'payments', 'plans', 'personal_access_tokens'
]
```

Y se generó un respaldo real de los datos vigentes (ver sección 4.4), cuyo conteo de documentos confirma que la base de datos contiene información real:

Colección	Documentos exportados
users	4
plans	1 (Básico ya editado por un administrador; Premium se sirve desde su valor por defecto)
contacts	0 (sin contactos activos al momento del corte)
alerts	0 (sin alertas activas al momento del corte)
payments	0 (sin pagos activos al momento del corte)

4.4 Procedimiento de respaldos

El respaldo se realiza con las herramientas oficiales de MongoDB, apuntando directamente a la cadena de conexión de Atlas:

1. **Respaldo completo restaurable**, con `mongodump` (excluyendo `personal_access_tokens`, credenciales de sesión temporales, no datos del negocio):

```
mongodump --uri="$MONGODB_URI" --excludeCollection=personal_access_tokens --out=dump/
```

Restauración:

```
mongorestore --uri="$MONGODB_URI" --db=alertsnc dump/alertsync
```

2. **Exportación legible en JSON**, con `mongoexport`, un archivo por colección:

```
mongoexport --uri="$MONGODB_URI" --collection=users --jsonArray --pretty --out=users.json
```

Ambos procedimientos son de **solo lectura** sobre la base de datos (no modifican ni borran información), por lo que pueden ejecutarse en cualquier momento sin riesgo. El resultado, ejecutado el 24 de julio de 2026, se incluye como evidencia en las carpetas `dump/` y `json/` de esta misma entrega.

Siguiente paso recomendado: exponer este mismo procedimiento como un botón "Generar respaldo" dentro del panel de administración web (protegido por el middleware `admin`), que ejecute `mongodump` en el servidor y ofrezca el archivo resultante como descarga directa al administrador — identificado como mejora planeada para la siguiente iteración del panel.

4.5 Índices necesarios para el proyecto

Colección	Índice	Tipo	Estado	Justificación
users	email	Simple (no único)	Aplicado	Acelera la búsqueda de cuentas por correo en login, activación y recuperación de contraseña. No es único porque varios administradores pueden compartir un correo de contacto.
users	username	Único, disperso	Aplicado	Identificador real de inicio de sesión para administradores. Disperso porque la mayoría de usuarios (rol user) no tienen username.
contacts	user_id + priority	Compuesto	Aplicado	Las consultas siempre filtran por usuario y ordenan por prioridad de notificación.
alerts	user_id + created_at (desc)	Compuesto	Aplicado	El historial de alertas siempre consulta por usuario, ordenado por fecha más reciente.
personal_access_tokens	token	Simple	Aplicado	Autenticación de cada request vía Sanctum: se busca el token exacto en cada petición.
payments	user_id + created_at (desc)	Compuesto	Identificado, pendiente de aplicar	El detalle de usuario en el panel admin y el estado de cuenta consultan siempre los pagos de un usuario ordenados por fecha.
plans	plan_id	Único	Identificado, pendiente de aplicar	Solo deben existir dos documentos (basico y premium); evita que una edición futura duplique un plan.

Los índices ya aplicados se verificaron directamente contra el clúster con `mongosh`:

```
> db.users.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { email: 1 }, name: 'email_1' },
  { v: 2, key: { username: 1 }, name: 'username_1', unique: true, sparse: true }
]
```

Los dos índices marcados como pendientes ya están definidos en el comando `php artisan mongodb:setup-indexes` (`app/Console/Commands/SetupMongoIndexes.php`) del backend, listos para ejecutarse en el siguiente despliegue.

4.6 Gestión de la calidad de los datos

Al no existir llaves foráneas, restricciones CHECK ni tipos de columna estrictos como en una base de datos relacional, MongoDB traslada buena parte de la responsabilidad de la calidad de los datos a la capa de aplicación. En ALERTSYNC esto se resuelve con:

- **Validación de entrada como allowlist:** cada endpoint usa `$request->validate([...])` de Laravel, que solo acepta los campos explícitamente declarados.

- **Tipos de dato controlados por el modelo (casts):** cada modelo Eloquent declara el tipo real de cada campo (boolean, integer, float, datetime, array).
- **Unicidad:** aplicada por índice a nivel de base de datos donde es absoluta (username), y a nivel de aplicación donde depende del contexto de negocio (correo de usuarios regulares).
- **Cifrado de contraseñas:** nunca se almacena una contraseña en texto plano; se usa bcrypt mediante el cast 'password' => 'hashed'.
- **Minimización de datos sensibles:** de una tarjeta de pago solo se guardan los últimos 4 dígitos y el nombre del titular.
- **Expiración explícita de datos temporales:** los códigos de activación y recuperación de contraseña llevan su propio campo `*_expires_at` y se invalidan inmediatamente después de usarse.
- **Integridad referencial a nivel de aplicación:** al eliminar un usuario se borran explícitamente sus contactos, alertas y pagos asociados.

5. Conclusiones

El proyecto ALERTSYNC utiliza MongoDB Atlas como motor de base de datos NoSQL en la nube, modelando la información en 6 colecciones de documentos con relaciones por referencia en lugar de llaves foráneas. Se documentó el esquema completo de cada colección, se generaron los archivos JSON de carga inicial para la única colección que realmente lo requiere (plans), se confirmó mediante mongosh que la base de datos está desplegada en Atlas y contiene datos reales, se ejecutó un respaldo completo (restaurable) y uno legible en JSON de la información vigente, y se identificaron y verificaron los índices necesarios para las consultas más frecuentes del sistema — dejando dos índices adicionales ya definidos en código y listos para aplicarse. Finalmente, se documentaron las medidas de calidad de datos adoptadas a nivel de aplicación, que en un modelo NoSQL sustituyen a las restricciones que en una base de datos relacional impondría el propio motor.

6. Bibliografía

- MongoDB, Inc. (2026). *MongoDB Manual — Documents*. <https://www.mongodb.com/docs/manual/core/document/>
- MongoDB, Inc. (2026). *MongoDB Atlas Documentation*. <https://www.mongodb.com/docs/atlas/>
- MongoDB, Inc. (2026). *MongoDB Manual — Indexes*. <https://www.mongodb.com/docs/manual/indexes/>
- MongoDB, Inc. (2026). *mongodump — MongoDB Database Tools*. <https://www.mongodb.com/docs/database-tools/mongodump/>
- MongoDB, Inc. (2026). *mongoexport — MongoDB Database Tools*. <https://www.mongodb.com/docs/database-tools/mongoexport/>
- MongoDB, Inc. (2026). *Laravel MongoDB Documentation*. <https://www.mongodb.com/docs/drivers/php/laravel-mongodb/>
- Laravel. (2026). *Laravel 13 Documentation — Eloquent: Getting Started*. <https://laravel.com/docs/eloquent>